

Course Code	Course Title	L	R	P	Cr
	AI-Assisted Problem Solving Using Python Programming				
Program Core					

Day 8 Session: Object-Oriented Programming & Data Exchange

Day 8

- Classes & Objects
- Inheritance & Polymorphism
- JSON & CSV Processing

Curiosity Question

How do professional software developers organize large applications and exchange data efficiently using Python?

Advantages

- Reusability
- Modularity
- Easy Maintenance
- Scalability

Why Object-Oriented Programming?

Traditional Programming

Functions, Variables

Object-Oriented Programming

Objects, Classes, Methods, Attributes

Real-Life Examples of Objects

Object	Attributes	Methods
Student	Name, Roll	Study(), Attend()
Car	Model, Color	Start(), Stop()
Mobile	Brand, RAM	Call(), Charge()

CLASS

A **class** is a blueprint used to create objects. It defines what data (variables) and actions (methods) an object will have.

Example

```
class Student:  
    name = "Pavan"  
    age = 20
```

Explanation of Example

- Student is a class
- name and age are variables inside the class
- No object is created yet → so no output

OBJECT

An **object** is an instance of a class. It represents a real entity.

Example

```
class Student:
    name = "Pavan"
    age = 20
```

```
s1 = Student()
```

```
print(s1.name)
print(s1.age)
```

Explanation

- s1 is an object of class Student
- We access data using object.variable
- Output:

Pavan

20

CONSTRUCTOR (`__init__`)

◆ A constructor is a special method that runs automatically when an object is created.

Example

```
class Student:
    def __init__(self):
        print("Object created")

s1 = Student()
```

Explanation

- `__init__` runs automatically
- When `s1` is created → message is printed

Output:

Object created

INSTANCE VARIABLES

- ◆ Variables created using `self` and belong to each object.

Example

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks
```

```
s1 = Student("Ravi", 80)
s2 = Student("Anu", 90)
```

```
print(s1.name, s1.marks)
print(s2.name, s2.marks)
```

Explanation

- Each object has its own values
- `s1` and `s2` store different data

Output:

```
Ravi 80
Anu 90
```

CLASS VARIABLES

Variables shared by all objects.

Example

```
class Student:  
    school = "ABC School"
```

```
s1 = Student()  
s2 = Student()
```

```
print(s1.school)  
print(s2.school)
```

Explanation

- `school` is same for all objects
- Stored once, shared by all

Output:

```
ABC School  
ABC School
```


INSTANCE METHOD

- ◆ A method that works with object data.

Example

```
class Student:
    def __init__(self, name):
        self.name = name

    def show(self):
        print("Name:", self.name)

s1 = Student("Ravi")
s1.show()
```

Explanation

- `show()` uses `self.name`
- Called using object

Output:

Name: Ravi

CLASS METHOD

- ◆ Works with class variables using `cls`.

Example

```
class Student:
    school = "ABC School"

    @classmethod
    def show_school(cls):
        print(cls.school)

Student.show_school()
```

Explanation

- No object needed
- Access class variable using `cls`

Output:

ABC School

STATIC METHOD

Does not use object or class data.

Example

```
class Demo:
    @staticmethod
    def greet():
        print("Welcome Students")
```

```
Demo.greet()
```

Explanation

- Independent method
- Used for utility functions

Output:

```
Welcome Students
```

ENCAPSULATION

◆ Encapsulation means **hiding data** to protect it.

PUBLIC VARIABLE

◆ Example

```
class Student:
    def __init__(self):
        self.name = "Ravi"
```

```
s = Student()
print(s.name)
```

Explanation

- Accessible anywhere
- No restriction

PROTECTED VARIABLE

◆ Example

```
class Student:
    def __init__(self):
        self._marks = 80
```

```
s = Student()
print(s._marks)
```

Explanation

- `_marks` is protected (just a convention)
- Should not be accessed outside (but possible)

PRIVATE VARIABLE

◆ Example

```
class Student:
    def __init__(self):
        self.__marks = 90

s = Student()
# print(s.__marks)    # Error
```

Explanation

- `__marks` is private
- Cannot be accessed directly

ACCESS PRIVATE VARIABLE (METHOD)

◆ Example

```
class Student:
    def __init__(self):
        self.__marks = 90

    def show(self):
        print(self.__marks)
```

```
s = Student()
s.show()
```

Explanation

- Access private variable through method
- Output:
90

GETTER AND SETTER

◆ Used to safely access and update private data.

Example

```
class Student:
    def __init__(self):
        self.__marks = 0

    def set_marks(self, m):
        self.__marks = m

    def get_marks(self):
        return self.__marks

s = Student()
s.set_marks(85)
print(s.get_marks())
```

Explanation

- `set_marks()` → update value
- `get_marks()` → read value

Output:

85

Example

```
class Bank:
    def __init__(self):
        self.__balance = 1000

    def deposit(self, amount):
        self.__balance += amount

    def show(self):
        print(self.__balance)
```

```
b = Bank()
b.deposit(500)
b.show()
```

Explanation

- Balance is private
- Access only through methods
- Output:
1500

◆ **Inheritance** allows one class to **reuse properties and methods** of another class.

☞ It helps in:

- Code reusability
- Reducing repetition
- Better organization

Example

```
class Animal:
    def eat(self):
        print("Animal eats food")

class Dog(Animal):
    pass

d = Dog()
d.eat()
```

Explanation

- Animal → Parent (Base Class)
- Dog → Child (Derived Class)
- Dog inherits eat() from Animal

Output:

Animal eats food

BASE CLASS (PARENT CLASS)

◆ A **Base Class** is the class whose properties are inherited by another class.

Example

```
class Vehicle:
    def start(self):
        print("Vehicle starts")
```

Explanation

- `Vehicle` is the base class
- It contains common functionality (`start()`)

DERIVED CLASS (CHILD CLASS)

◆ A **Derived Class** is the class that inherits from another class.

Example

```
class Vehicle:
    def start(self):
        print("Vehicle starts")

class Car(Vehicle):
    def drive(self):
        print("Car is driving")

c = Car()
c.start()
c.drive()
```

Explanation

- Car inherits from Vehicle
- Can use both:
 - inherited method → start()
 - its own method → drive()

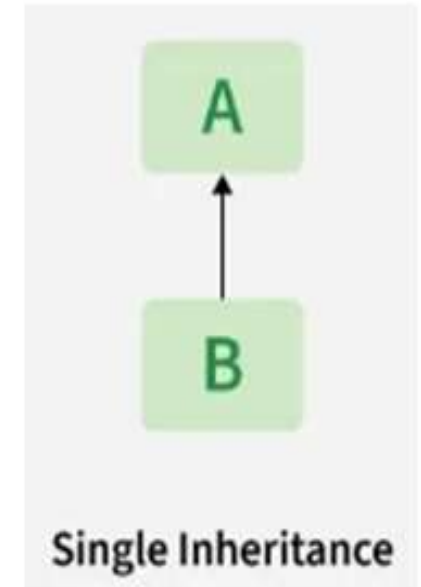
Output:

```
Vehicle starts
Car is driving
```

TYPES OF INHERITANCE

- ◆ **1. Single Inheritance:** In Single Inheritance one class extends another class (one class only).
One parent → one child

```
#single Inheritance: parent to child
class parent:
    def p_display(self):
        print("Parent Class")
class child(parent):
    def c_display(self):
        print("Child Class")
c1 = child()
c1.p_display()
c1.c_display()
```

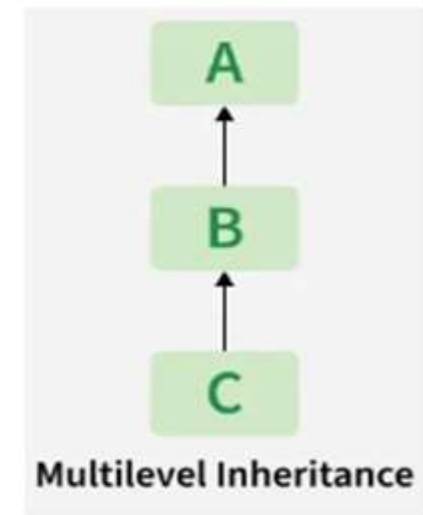


2. Multilevel Inheritance : $A \rightarrow B \rightarrow C$ (chain)

Multilevel inheritance is a type of inheritance where a class inherits from another derived class. This forms a chain of inheritance.

```
#Multi-level Inheritance:
class grandparent:
    def gp_display(self):
        print("Grand Parent Class")
class parent(grandparent):
    def p_display(self):
        print("Parent Class")
class child(parent):
    def c_display(self):
        print("Child Class")

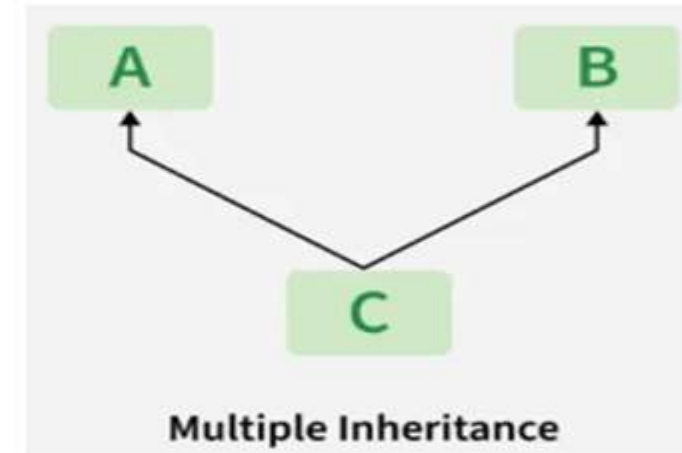
c1 = child()
c1.gp_display()
c1.p_display()
c1.c_display()
```



Multiple Inheritance : In Multiple Inheritance, one class extending more than one class.

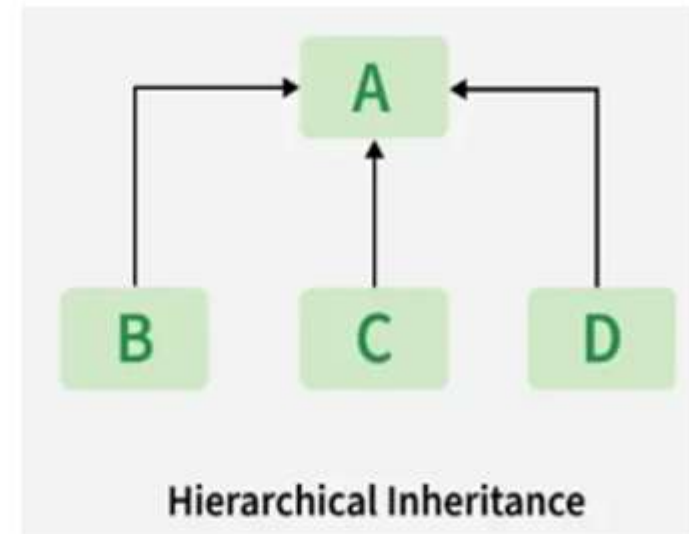
```
#Multiple Inheritance : Two Parents (Father,Mother) derives child class
class father:
    def f_display(self):
        print("Father Class")
class mother:
    def m_display(self):
        print("Mother Class")
class child(father,mother):
    def c_display(self):
        print("Child Class")

c1 = child()
c1.f_display()
c1.m_display()
c1.c_display()
```



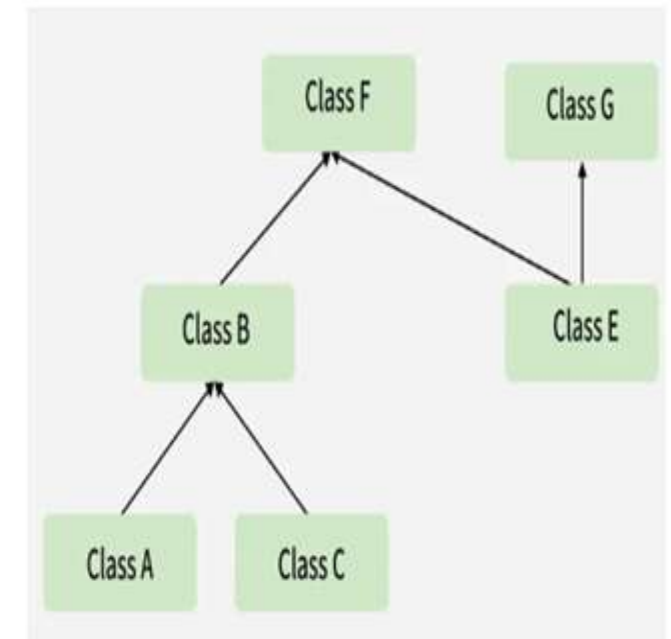
In Hierarchical Inheritance, one class is inherited by many sub classes.

```
#Hierarchical Inheritance: one parent two or more childs
class parent:
    def p_display(self):
        print("Parent Class")
class child1(parent):
    def c1_display(self):
        print("Child1 Class")
class child2(parent):
    def c2_display(self):
        print("Child2 Class")
#child1 Object Creation
c1 = child1()
c1.p_display()
c1.c1_display()
#child2 object Creation
c2 = child2()
c2.p_display()
c2.c2_display()
```



Hybrid inheritance is a combination of more than one type of inheritance. It uses a mix like single, multiple, or multilevel inheritance within the same program.

```
#Hybrid Inheritance
# Base class
class School:
    def func1(self):
        print("This function is in school.")
# Derived class 1 (Single Inheritance)
class Student1(School):
    def func2(self):
        print("This function is in student 1.")
# Derived class 2 (Another Single Inheritance)
class Student2(School):
    def func3(self):
        print("This function is in student 2.")
# Derived class 3 (Multiple Inheritance)
class Student3(Student1, School):
    def func4(self):
        print("This function is in student 3.")
# Driver code
obj = Student3()
obj.func1()
obj.func2()
```



USING `super()` (Access Parent Method)

◆ `super()` is used to call parent class method.

Example

```
class Animal:
    def sound(self):
        print("Animal sound")
```

```
class Dog(Animal):
    def sound(self):
        super().sound()
        print("Dog barks")
```

```
d = Dog()
d.sound()
```

Explanation

- Calls parent method first
- Then child method

Output:

Animal sound

Dog barks

POLYMORPHISM

◆ **Polymorphism** means:

☞ **Same method name, different behavior**

TYPES OF POLYMORPHISM

◆ 1. Method Overloading (Using Default Arguments)

Python does not support true overloading, but we can simulate it.

Example

```
class Math:
    def add(self, a, b=0, c=0):
        print(a + b + c)
```

```
m = Math()
m.add(5, 10)
m.add(5, 10, 15)
```

Explanation

- Same method handles different inputs

Output:

15

30

Operator Overloading

☞ Same operator behaves differently

Example

```
print(5 + 5)           # addition
print("Hi " + "All")   # string join
```

Explanation

- + works for numbers and strings differently

Output:

10

Hi All

Polymorphism with Functions

```
def display(a):  
    print(a)
```

```
display(10)  
display("Hello")  
display([1,2,3])
```

Explanation

- Same function works for different data types

```
class Shape:
    def area(self):
        print("Area formula")
```

```
class Circle(Shape):
    def area(self):
        print("Area =  $\pi r^2$ ")
```

```
class Square(Shape):
    def area(self):
        print("Area =  $side^2$ ")
```

```
c = Circle()
s = Square()
```

```
c.area()
s.area()
```

Explanation

- Same method `area()` behaves differently
- Example of polymorphism

METHOD OVERRIDING

◆ **Method Overriding** means:

☞ Child class **redefines a method** of the parent class.

Example

```
class Animal:
    def sound(self):
        print("Animal makes sound")
```

```
class Dog(Animal):
    def sound(self):
        print("Dog barks")
```

```
d = Dog()
d.sound()
```

Explanation

- Parent method is replaced by child method
- Dog overrides sound()

Output:

Dog barks

```
#polymorphism : Implementing same method in different context
#methods with same name implementing in different way
#Polymorphism is achieved by two ways Overloading, Overriding
#1. Overloading :
'''1. Operator Overloading : + - addition, + - Concatination
2. Method Overloading : add(), add(10,20)'''
#Example:Method Overloading
class moverload:
    def add(self,a=None,b=None):
        print(a,b)
obj = moverload()
obj.add()
obj.add(10)
obj.add(10,20)

#2. Overriding: can be implemented with inheritance
'''Method name should be same, number of arguments should be same'''
class father:
    def mobile(self):
        print("Father Mobile Nokia")
class child:
    def mobile(self):
        print("Child Mobile Iphone")
obj = father()
obj = child()
obj.mobile()
```


ABSTRACTION

Abstraction means **hiding implementation details** and showing only essential features to the user.

☞ In simple words:

User knows *what to do*, not *how it works internally*.

How to Achieve Abstraction in Python?

Using:

- Abstract classes
- abc module

Example

```
from abc import ABC, abstractmethod

class Vehicle(ABC):

    @abstractmethod
    def start(self):
        pass

class Car(Vehicle):
    def start(self):
        print("Car starts with key")

c = Car()
c.start()
```

Explanation

- ABC → Abstract Base Class
- start() → abstract method (no implementation in parent)
- Car must implement start()
- If not implemented → error

Output:

Car starts with key

Key Points

- Cannot create object of abstract class
- Must override abstract methods
- Used for **design structure**

```
#Abstraction:
'''Hiding the Implementation details and
Showing the Essential details is called Abstraction
Example : apk files, exe files...'''
#abstract class, abstract method, concrete class
#abstract class :which have abstract methods
#abstract method : method with declaration with no defination
#concrete class : class without abstract methods

#import from module ABC,abstractmethod
from abc import ABC,abstractmethod
class AbstractDemo(ABC):
    @abstractmethod
    def display(self):
        None
    @abstractmethod
    def show(self):
        None
class Demo(AbstractDemo):
    def display(self):
        print("Abstract Method")
    def show(self):
        print("show Method")
class Demo1(AbstractDemo):
    def show(self):
        print("Show Method")
    def display(self):
        print("Display Method")
obj = Demo1()
obj.display()
```

Introduction to JSON

- JSON = **JavaScript Object Notation**
- A lightweight format used for **data storage and data exchange**
- Commonly used in:
 - Web APIs
 - Data transfer between server & client
 - Configuration files

◆ Key Features

- Human-readable
- Easy to parse by machines
- Language-independent

JSON (JavaScript Object Notation)

is a lightweight, text-based data format used to store and exchange data.

It represents data in key–value pairs and ordered lists.

Example JSON:

File Name: data.json

```
{  
  "name": "SR University",  
  "age": 22,  
  "course": "CSE",  
  "marks": [85, 90, 88]  
}
```

#Reading JSON Data using json Module

Python provides the built-in json module to work with JSON data.

```
import json
```

#1. json.load() – Reading JSON from a File

#Reads JSON data from a file and converts it into a Python dictionary.

```
import json
```

```
f = open("data.json", "r")
```

```
data = json.load(f)
```

```
print(data)
```

```
f.close()
```

#2. json.loads() – Reading JSON from a String

#Reads JSON data from a string.

```
import json
```

```
s = '{"name": "SRU", "age": 22}'
```

```
data = json.loads(s)
```

```
print(data)
```

#Writing JSON Data using json Module

#1. json.dump() – Writing JSON to a File

#Writes Python data into a JSON file.

```
import json
```

```
data = {  
    "name": "Vijay Kumar",  
    "id": 3022,  
    "subject": "PSPP",  
}
```

```
f = open("output.json", "w")
```

```
json.dump(data, f)
```

```
f.close()
```

#2. json.dumps() – Writing JSON to a String

#Converts Python data into a JSON-formatted string.

```
import json
```

```
data = {"name": "SRU", "age": 22}
```

```
s = json.dumps(data)
```

```
print(s)
```

#2. Working with CSV Data

```
import csv
```

CSV Format

CSV (Comma Separated Values) stores tabular data in plain text form, where each line represents a row and values are separated by commas.

Example CSV File:

Name	Age	Course
SRU	22	PSPP
SRIT	25	COA
SRIX	26	DBMS

#Reading CSV Data using csv Module

Python provides the csv module for handling CSV files.

1. csv.reader – Reading CSV Data as Lists

Reads each row as a list.

```
import csv  
  
f = open("data1.csv", "r")  
r = csv.reader(f)  
  
for row in r:  
    print(row)  
  
f.close()
```

#2. Working with CSV Data

```
import csv
```

CSV Format

CSV (Comma Separated Values) stores tabular data in plain text form, where each line represents a row and values are separated by commas.

#Writing CSV

```
#Method : csv.writer()
```

```
import csv
```

```
f = open("data2.csv", "w", newline="")
```

```
writer = csv.writer(f)
```

```
writer.writerow(["name", "age"])
```

```
writer.writerow(["SRU", 6])
```

```
writer.writerow(["SREC", 18])
```

```
writer.writerow(["SRIX", 10])
```

```
f.close()
```




Thank you